eg. Student class

```
class Student :

    def __init__(self, name, age):

        self.name = name
        self.age  = age

    def display_info(self):

        print("Student Name is:",
                        self.name)
        print("Student age is:",
                        self.age)

S1 = Student("Rahul", 20)
S2 = Student("Akshay", 25)
print(S1.age)
S1.display_info()
```

Method vs Functions

Method	Functions

1) Define inside the class

2) Call through object

3) Call Syntax :

obj-name . method-name()

4) exicte self

5) eg. list :-

append()
remove()
insert()
copy()
count()

1) Define outside the class

2) Call directly.

3) Call Syntax :

method-name()

4) Does not exists self

5) Built in fun :

len(), min(), max(), sum(), type(), id()

class list :

def append(self):

def remove(self):

class dict :

add

class set :

add()

→ def len()

eg. list1 = list() → def sum()

→ def min()

list1.append(10)

print(10)

o/p - [10]

list1 = [10, 20, 30] ← 20

list2.append(20)

Constructor - __init__ method

class:

def sum_calculate(self): ← normal method

obj.sum_calculate() ← fun call

def --init--(self):

init method or Constructor

obj1 = class1()

① --init-- is the only method which executed automatically, when we create an object of our class.

This is the Superpower of Constructor.

② But we have to call the other methods explicitly.

Normal method	--init-- method /Constructor
① explicitly call	① when we create an object

automatically calls it.

② obj1.method_name()

② obj1 = class_name()

③ User has control

④ No user control

Constructor – we write configuration related code inside the constructor that we want to execute automatically without user action.

1] Database connection code

2] API connection setup.

*self is ... self is the reference

self :- self is the reference of current object of the class.

There are two components in class

1) attributes, properties, data
2) methods

we can access it only through object.

```
print(obj.attribute_name)
obj.method_name()
```

we use self in 3 places

1) method definition :-

```
class Vehical:
```

1st parameter

def __init__ (self):

def display_info(self):

2] Attribute Name :-

class Vehical:

def __init__ (self, color, speed):

self.color = color
self.speed = speed

self

} class Variable

3] Method calls inside class

class Vehical:

def __init__ (self, speed):
    self.speed = speed

def method1(self):

① ② ③

```
def method2(self):
    self.method1()
```